



UNIVERSITETI / UNIVERSITY
"ISA BOLETINI"
MITROVICË

Big Data Processing

...

PYSPARK RDD, DATA FRAMES AND SQL EXERCISES

Example 9: DataFrame and PySpark SQL Program - PART 1

Every 4 years, soccer fans throughout the world celebrate a festival called “Fifa World Cup” and with that, everything seems to change in many countries. In this 3 part exercise, you'll be doing some exploratory data analysis (EDA) on the "FIFA 2018 World Cup Player" dataset using PySpark SQL which involves DataFrame operations, SQL queries, and visualization.

In the first part, you'll load FIFA 2018 World Cup Players dataset (Fifa2018_dataset.csv), which is in CSV format, into a PySpark's DataFrame and inspect the data using basic DataFrame operations.

Remember, you already have a SparkSession spark and a variable file_path available in your workspace.

Instructions:

- Create a PySpark DataFrame from file_path (which is the path to the Fifa2018_dataset.csv file).
- Print the schema of the DataFrame.
- Print the first 10 observations.
- How many rows are in there in the DataFrame?

```

1 # Load the Dataframe
2 fifa_df = spark.read.csv(file_path, header=True, inferSchema=True)
3
4 # Check the schema of columns
5 fifa_df.printSchema()
6
7 # Show the first 10 observations
8 fifa_df.show(10)
9
10 # Print the total number of rows
11 print("There are {} rows in the fifa_df DataFrame".format(fifa_df.count()))

```

root

```

|-- _c0: integer (nullable = true)
|-- Name: string (nullable = true)
|-- Age: integer (nullable = true)
|-- Photo: string (nullable = true)
|-- Nationality: string (nullable = true)
|-- Flag: string (nullable = true)
|-- Overall: integer (nullable = true)
|-- Potential: integer (nullable = true)
|-- Club: string (nullable = true)
|-- Club Logo: string (nullable = true)
|-- Value: string (nullable = true)
|-- Wage: string (nullable = true)
|-- Special: integer (nullable = true)
|-- Acceleration: string (nullable = true)

```

_c0	Name	Age	Photo	Nationality	Flag	Overall	Potential
0	Cristiano Ronaldo	32	https://cdn.sofif...	Portugal	https://cdn.sofif...	94	94
1	L. Messi	30	https://cdn.sofif...	Argentina	https://cdn.sofif...	93	93
2	Neymar	25	https://cdn.sofif...	Brazil	https://cdn.sofif...	92	94
3	L. Suárez	30	https://cdn.sofif...	Uruguay	https://cdn.sofif...	92	92
4	M. Neuer	31	https://cdn.sofif...	Germany	https://cdn.sofif...	92	92
5	R. Lewandowski	28	https://cdn.sofif...	Poland	https://cdn.sofif...	91	91
6	De Gea	26	https://cdn.sofif...	Spain	https://cdn.sofif...	90	92
7	E. Hazard	26	https://cdn.sofif...	Belgium	https://cdn.sofif...	90	91
8	T. Kroos	27	https://cdn.sofif...	Germany	https://cdn.sofif...	90	90
9	G. Higuain	29	https://cdn.sofif...	Argentina	https://cdn.sofif...	90	90

only showing top 10 rows

There are 17981 rows in the fifa_df DataFrame

Example 9: DataFrame and PySpark SQL Program - PART 2

The `fifa_df` DataFrame that we created has additional information about datatypes and names of columns associated with it. This additional information allows PySpark SQL to run SQL queries on DataFrame. SQL queries are concise and easy to run compared to DataFrame operations. But in order to apply SQL queries on DataFrame first, you need to create a temporary view of DataFrame as a table and then apply SQL queries on the created table (Running SQL Queries Programmatically).

In the second part, you'll create a temporary table of `fifa_df` DataFrame and run SQL queries to extract the 'Age' column of players from Germany.

You already have a `SparkContext` `spark` and `fifa_df` available in your workspace.

Instructions:

- Create temporary table `fifa_df_table` from `fifa_df` DataFrame.
- Construct a "query" to extract the "Age" column from Germany players in `fifa_df_table`.
- Apply the SQL "query" and create a new DataFrame `fifa_df_germany_age`.
- Computes basic statistics of the created DataFrame.

```
1 # Create a temporary view of fifa_df
2 fifa_df.createOrReplaceTempView('fifa_df_table')
3
4 # Construct the "query"
5 query = '''SELECT Age FROM fifa_df_table WHERE Nationality == "Germany"'''
6
7 # Apply the SQL "query"
8 fifa_df_germany_age = spark.sql(query)
9
10 # Generate basic statistics
11 fifa_df_germany_age.describe().show()
```

```
+-----+-----+
|summary|      Age|
+-----+-----+
|  count|      1140|
|   mean|24.20263157894737|
| stddev|4.197096712293756|
|   min|         16|
|   max|         36|
+-----+-----+
```

Example 9: DataFrame and PySpark SQL Program - PART 3

Data visualization is important for exploratory data analysis (EDA). PySpark DataFrame is perfect for data visualization compared to RDDs because of its inherent structure and schema.

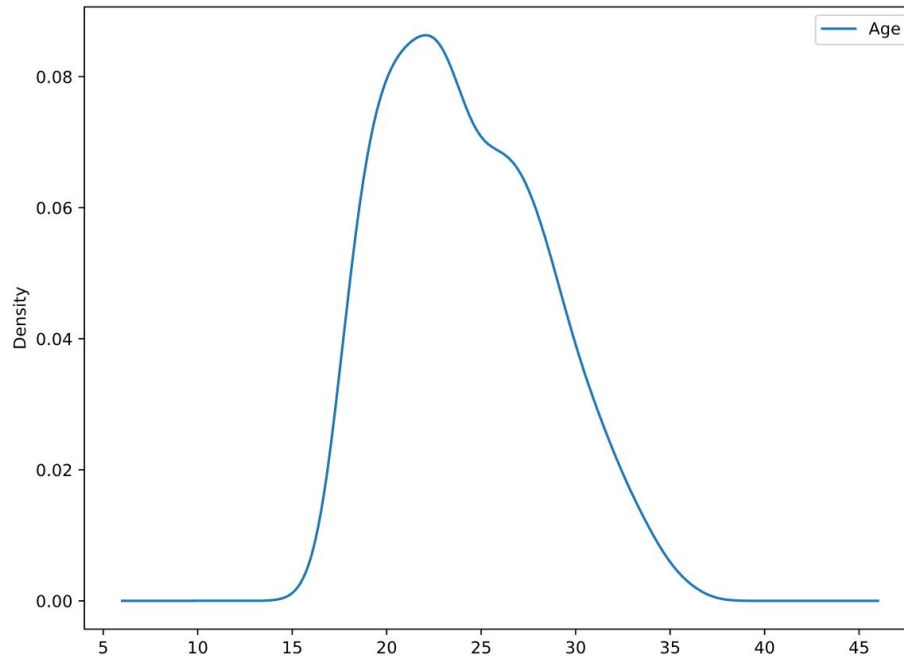
In this third part, you'll create a histogram of the ages of all the players from Germany from the DataFrame that you created in the previous exercise. For this, you'll first convert the PySpark DataFrame into Pandas DataFrame and use matplotlib's `plot()` function to create a density plot of ages of all players from Germany.

Remember, you already have a `SparkSession` `spark`, a temporary table `fifa_df_table` and a DataFrame `fifa_df_germany_age` available in your workspace.

Instructions:

- Convert `fifa_df_germany_age` to `fifa_df_germany_age_pandas` Pandas DataFrame.
- Generate a density plot of the 'Age' column from the `fifa_df_germany_age_pandas` Pandas DataFrame.

```
1 # Convert fifa_df to fifa_df_germany_age_pandas DataFrame
2 fifa_df_germany_age_pandas = fifa_df_germany_age.toPandas()
3
4 # Plot the 'Age' density of Germany Players
5 fifa_df_germany_age_pandas.plot(kind='density')
6 plt.show()
```



Example 10: 4 Parts to be done, base code given.

```
1  from pyspark.sql import SparkSession
2
3  spark = SparkSession.builder.appName("Example10").getOrCreate()
4  sc = spark.sparkContext
5
6  employees_data = [
7      (1, "Alice", "Engineering", 90000, 5, "Berlin"),
8      (2, "Bob", "Engineering", 80000, 3, "Berlin"),
9      (3, "Charlie", "HR", 60000, 4, "Munich"),
10     (4, "Diana", "HR", 65000, 6, "Munich"),
11     (5, "Eve", "Finance", 70000, 7, "Hamburg"),
12     (6, "Frank", "Finance", 72000, 4, "Berlin"),
13     (7, "Grace", "Engineering", 95000, 8, "Hamburg"),
14     (8, "Heidi", "Marketing", 55000, 2, "Berlin"),
15     (9, "Ivan", "Marketing", 58000, 3, "Hamburg"),
16     (10, "Judy", "Engineering", 99000, 10, "Munich"),
17 ]
18
19 employees_columns = ["id", "name", "dept", "salary", "years_exp", "city"]
20
21 employees_df = spark.createDataFrame(employees_data, employees_columns)
22 employees_df.show()
23 employees_df.printSchema()
24
25 employees_df.createOrReplaceTempView("employees")
26
```


Example 10 - Part 1

Department average salary (RDD → DataFrame → SQL)

Task description:

1. Convert employees_df to an RDD.
2. From the RDD, create pairs (dept, (salary, 1)).
3. Use reduceByKey to compute total salary and count of employees per department.
4. Compute the average salary per department in the RDD.
5. Convert the result to a DataFrame dept_avg_df with columns ("dept", "avg_salary").
6. Register it as a temporary view and use SQL to select the top 3 departments by avg_salary (descending).

```
# === Exercise 1: Department average salary ===

# Step 1: RDD from DataFrame
employees_rdd = employees_df.rdd

# Step 2: Create (dept, (salary, 1)) pairs
# TODO: replace None with the correct transformation
dept_salary_rdd = employees_rdd.map(lambda row: None)

# Step 3: Use reduceByKey to sum salary and count
# TODO: implement the lambda to add up salaries and counts
dept_sum_count_rdd = dept_salary_rdd.reduceByKey(lambda a, b: None)

# Step 4: Compute (dept, avg_salary)
# TODO: map to (dept, average_salary)
dept_avg_rdd = dept_sum_count_rdd.map(lambda kv: None)

# Step 5: Convert to DataFrame and show
dept_avg_df = dept_avg_rdd.toDF(["dept", "avg_salary"])
dept_avg_df.show()

# Step 6: Register as view and write a SQL query to get top 3 departments
dept_avg_df.createOrReplaceTempView("dept_avg")

top3_sql = spark.sql("""
    -- TODO: write the SQL query here
""")

top3_sql.show()
```

Example 10 - Part 2

Employees above department average (DF + SQL + RDD)

Task description:

1. Using DataFrame operations, compute average salary per department (column name avg_salary).
2. Join this DataFrame with employees_df on department.
3. Filter to keep only employees whose salary is greater than avg_salary.
4. Use SQL on the joined DataFrame to select name, dept, salary, avg_salary sorted by department and salary (descending).
5. Convert the filtered DataFrame to an RDD and compute, for each department, how many employees are above average.

```

3 # Step 1: DataFrame with average salary per dept
4 dept_avg_df = (
5     employees_df
6     .groupBy("dept")
7     # TODO: aggregate to get avg salary with alias "avg_salary"
8 )
9
10 dept_avg_df.show()
11
12 # Step 2: Join employees with dept averages
13 # TODO: join on "dept"
14 joined_df = employees_df.join(dept_avg_df, on="dept", how="inner")
15
16 # Step 3: Filter employees with salary > avg_salary
17 # TODO: use a filter condition with F.col
18 high_earners_df = joined_df # replace with filtered DataFrame
19
20 high_earners_df.show()
21
22 # Step 4: Use SQL to show same result
23 joined_df.createOrReplaceTempView("employees_with_avg")
24
25 high_earners_sql = spark.sql("""
26     -- TODO: write SQL to select name, dept, salary, avg_salary
27     -- and filter salary > avg_salary, ordered by dept, salary DESC
28 """)
29
30 high_earners_sql.show()
31
32 # Step 5: Convert high_earners_df to RDD and count per dept
33 high_earners_rdd = high_earners_df.rdd
34
35 # TODO: map to (dept, 1) and use reduceByKey to count
36 dept_high_count_rdd = high_earners_rdd # replace with your code
37
38 for dept, cnt in dept_high_count_rdd.collect():
39     print(dept, cnt)

```

Example 10 - Part 3

Seniority categories (RDD → DataFrame → SQL)

Task description:

1. From `employees_df.rdd`, create an RDD of (name, dept, years_exp, category) where:
 - a. < 3 years → "junior"
 - b. 3–6 years → "mid"
 - c. > 6 years → "senior"
2. Convert this RDD to a DataFrame with columns ("name", "dept", "years_exp", "category").
3. Register it as a temp view `seniority`.
4. Use SQL to count how many employees per dept and category.
5. Order the result by dept and category.

```
3 # Step 1: Build RDD with category
4 def categorize(row):
    Implement with Codex
5     # TODO: implement the logic for "junior", "mid", "senior"
6     # based on row.years_exp
7     category = None
8     return (row.name, row.dept, row.years_exp, category)
9
10 seniority_rdd = employees_df.rdd.map(categorize)
11
12 # Step 2: Convert to DataFrame
13 seniority_df = seniority_rdd.toDF(["name", "dept", "years_exp", "category"])
14 seniority_df.show()
15
16 # Step 3: Register as view and write SQL query
17 seniority_df.createOrReplaceTempView("seniority")
18
19 counts_sql = spark.sql(""
20     -- TODO: count employees per dept and category,
21     -- and order by dept, category
22     "")
23
24 counts_sql.show()
```

Example 10 - Part 4

Department budget utilization (RDD → DF → SQL)

Task description:

1. Define a list of (dept, budget) in Python (e.g. Engineering, HR, Finance, Marketing).
2. Parallelize this list into an RDD and convert it to a DataFrame budget_df with columns ("dept", "budget").
3. Join employees_df with budget_df on dept.
4. Use SQL to compute, for each department:
 - a. total salary (SUM(salary) as total_salary)
 - b. budget (MAX(budget) as budget)
 - c. utilization ratio = total_salary / budget (round to 2 decimals).
5. Order by utilization_ratio (descending).


```
3 # Step 1: Hardcoded budget data
4 budget_data = [
5     # TODO: fill in tuples like ("Engineering", 400000)
6 ]
7
8 # Step 2: RDD -> DataFrame
9 budget_rdd = sc.parallelize(budget_data)
10 budget_df = budget_rdd.toDF(["dept", "budget"])
11 budget_df.show()
12
13 # Step 3: Join employees with budgets
14 emp_with_budget_df = employees_df.join(budget_df, on="dept", how="inner")
15 emp_with_budget_df.createOrReplaceTempView("emp_budget")
16
17 # Step 4: Write a SQL query to compute total_salary, budget, utilization_ratio
18 utilization_sql = spark.sql("""
19     -- TODO:
20     -- SELECT dept,
21     --         SUM(salary) AS total_salary,
22     --         MAX(budget) AS budget,
23     --         ROUND(SUM(salary) / MAX(budget), 2) AS utilization_ratio
24     -- FROM emp_budget
25     -- GROUP BY dept
26     -- ORDER BY utilization_ratio DESC
27 """)
28
29 utilization_sql.show()
30
```



UNIVERSITETI / UNIVERSITY
"ISA BOLETINI"
MITROVICË

Stay Curious. Questions?

Thank you for your attention.
Lorent Sinani



Scan to connect on LinkedIn